zcring

Zero-copy shared-memory IPC for embedded Linux

One producer publishes once. N consumers read the same bytes in place.
No memcpy and no system call on the data path.

SSM / C-DAC Next-Gen Kernel Hackathon · Track 1

Problem statement: Zero-Copy Shared-Memory IPC Framework for Embedded Linux

C11 · no external dependencies · ThreadSanitizer clean · every figure traceable to committed raw data

The cost is copies — and worse, it is unpredictable

Conventional Linux IPC moves every message through the kernel twice

Four passes over memory per message.

The producer writes it, the kernel copies it in, the kernel copies it out, the consumer reads it.
Two of those four are pure overhead.

A system call on every send and every receive.

At small message sizes this, not the copying, is what dominates.

For real-time embedded work the variance matters more than the mean.

A control loop that is usually fast and occasionally late is a control loop that is late.

Sensor → controller → actuator pipelines need an answer on time, every time.
Determinism is the requirement; throughput is a side effect.

Share the memory instead of copying through it

A lock-free MPMC ring over a memfd-backed mapping

passes over memory, per message	zcring	pipe / unix socket
producer writes payload	1	1 (into a staging buffer)
copy into kernel	—	1
copy out of kernel	—	1
consumer reads payload	1	1
total	**2**	**4**

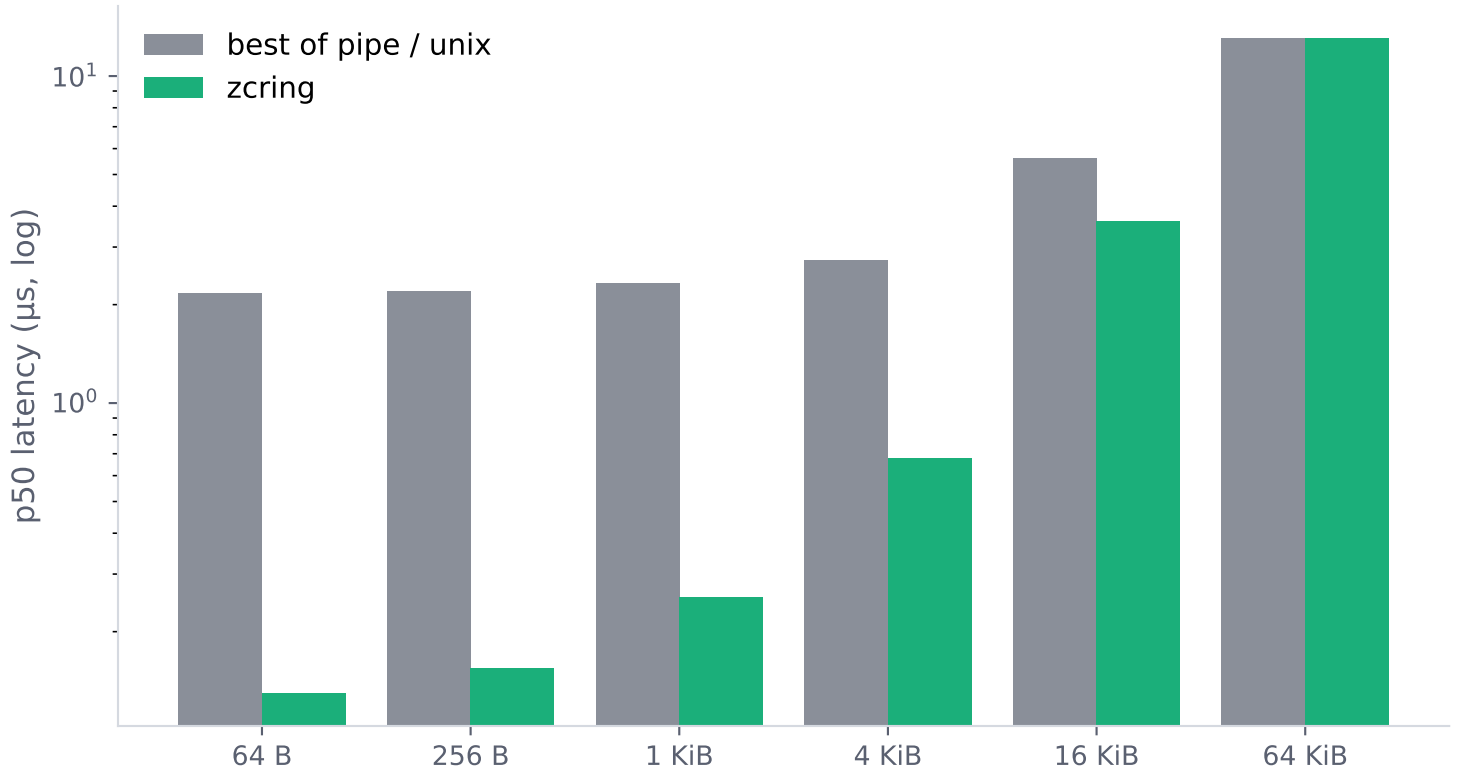
reserve() / commit() and acquire() / release() hand back raw pointers into shared memory.

The application constructs and reads messages in place. There is no memcpy anywhere in the data path.

Architecture diagram submitted separately · Vyukov sequence scheme · head and tail on separate cache lines

Small messages: 16.8x lower latency

One-way p50, bare metal, dual-core + SMT, every payload byte written and read



	zcring p50	vs best
64 B	130 ns	**16.8x**
1 KiB	254 ns	**9.1x**
4 KiB	677 ns	**4.0x**
16 KiB	3.6 us	**1.56x**

Measured 20 Aug against the code in this repo, and reproduced by a second independent run to three significant figures. Earlier drafts quoted 19.6x from the Layer 1 revision; a later code change cost ~16 ns per small message and the historical dataset is kept, labelled.

Dedicated-core (polling) configuration — see next slide. Raw data: results/sweep.csv

Two deployment postures, stated up front

The small-message win comes from removing the syscall — so it depends on not making one

Dedicated core (spin)

the consumer polls the ring

- 64 B: 130 ns · 16.8x vs pipe
- lowest achievable latency
- costs one core, continuously
- standard practice in real-time embedded

Shared core (adaptive notify)

the consumer blocks when blocking pays

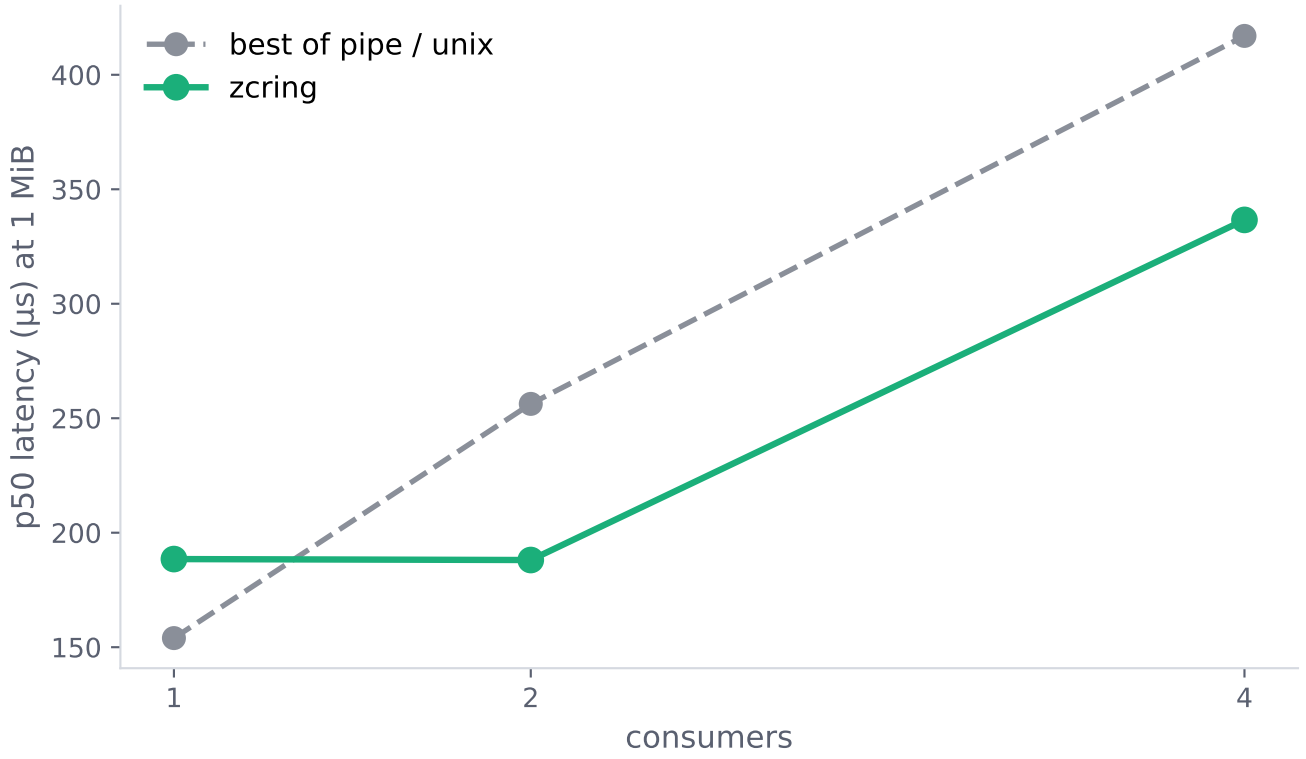
- 64 B: 2.3 µs · ~parity with pipe
- idle CPU cost comparable to a socket
- syscall returns, so the small-message edge goes
- the copy advantage is unaffected

Both are shipped. The framework does not pick for you — the deployment does.

The 16.8x headline is a dedicated-core number and is labelled as such everywhere it appears. Measured 20 Aug against the code in this repo.

Scalability: publication is O(1) in consumer count

One release store regardless of N — copying transports pay N copies each way



1 MiB payload, adaptive notify

N = 1	**0.82×**
N = 2	**1.36×**
N = 4	**1.24×**

zcring loses at one consumer and wins from two onward. The crossover is the result — not a flat multiplier.

N=4 on two physical cores is oversubscribed — four consumers wake simultaneously onto two cores.
Not quoted as a scalability result in either direction; it needs a machine with more cores to measure honestly.

Determinism: we found the jitter and removed it

Tail latency traced to a hardware cause, not tuned away

Deep CPU idle states on this part:

POLL	0 µs
C1	1 µs
C2	253 µs
C3	**1048 µs**

C3's 1048 µs exit latency matched an observed 1220 µs p99.9 almost exactly.

p99.9 at 64 B, 5 repetitions

idle states enabled	**354 µs**
range across reps	787 ns – 1.77 ms
deep idle disabled	**2.4 µs**
range across reps	1.4 – 4.2 µs

The mean dropping is not the point. The variance collapsing is.

p99.99 still shows a smaller, separate source (IRQ / scheduling jitter). Disclosed, and the target of the isolcpus / PREEMPT_RT work.

The notification threshold is learned, not configured

An online-learned policy where a fixed constant cannot work

The decision.

A consumer waiting on an empty ring must choose a spin budget S before observing the idle gap. Spin too long and CPU is wasted; spin too little and every message pays the block-and-wake cost.

The objective is constrained, not scalarised.

Minimise expected added latency W(1-F(S)) subject to expected spin cost <= beta*E[X]. Beta - the fraction of an idle gap the deployment will spend spinning - is the only input. Everything else is measured.

Estimator.

Robbins-Monro stochastic-approximation quantile, multiplicative step, two shifts and an add. Scale-free across decades of message rate, and deliberately non-annealing.

Bias correction.

Waits that end in a block are censored samples. The producer stamps the wake time, which recovers the true arrival and stops the estimator being pushed toward spinning by the very outcome that should not do that.

Floor.

Ski-rental: S >= measured wake cost. Rules out the one unambiguously wrong call - blocking below break-even. It does not make the policy 2-competitive, and we do not claim that.

Exploration.

Epsilon-greedy: the greedy policy censors its own observations and cannot tell a gap just past S from one far past it. Cost bounded, charged to the same budget.

Model type: inbuilt. Measured convergence at 100 µs pacing: gap estimate 97.8 µs, wake cost 2.07 µs, exploration 0.82% against a nominal 0.78%. At 1 MiB the same estimator learned 185 µs — the real inter-arrival, not the nominal pacing. That is the difference between measuring and configuring.

How we know the numbers mean anything

Six ways this benchmark could have lied, found and corrected

A harness that never touched the payload.

The first result was a perfect flat line, because only an 8-byte header moved. Every payload cache line is now written and read.

SMT-sibling pinning.

Two roles on one physical core left p50 looking fine and inflated p99 by 185x. The scripts now read the topology and refuse to do it.

Deep C-state entry.

A 1048 us idle-state exit latency landing in the tail. See previous slide.

Thermal throttling.

Sustained benchmarking heat-soaked a low-TDP part; large payloads degraded 70%. Every run is now gated on package temperature.

A saturating offered rate.

A 64 MiB ring against a 200 KB socket buffer at saturation measures queueing, not transport. Rate is now a stated fraction of measured saturation.

Orphaned consumers.

The textbook getppid()==1 check silently fails under a subreaper. Found by killing the producer and watching, not by reading code.

Whole suite re-measured on a separate occasion from a clean boot: the fan-out result reproduced within 1.5%.
Every figure in this deck is generated from committed CSVs by a script in the repository.

What it enables

Camera → inference + display + recorder, one producer feeding three consumers

640×480 frames at 30 fps, three consumers, ten minutes, run against an identical pipeline over three UNIX sockets.

frames delivered	frame rate, zero drift	frames dropped	memory traffic avoided
18 000/18 000	**30.000 fps**	**0**	**30.9 GB**

Three consumers read the same frame. A copying transport would move it three times.

make demo · runs unattended for ten minutes · resident memory flat, verified shared rather than duplicated via smaps_rollup

What it does not do, and what comes next

Stated by us rather than found by you

Single-consumer large payloads are slower than a UNIX socket.

0.82x at 1 MiB, under the determinism-first configuration this project requires. Offered rate, thermal state and busy-spin were each eliminated as causes; the remaining candidates are platform power and frequency properties. Fan-out recovers the case from N=2.

Consumer count is bounded by core count, on any platform.

In broadcast mode every consumer runs on every message, so one producer plus four consumers oversubscribes a four-core embedded target exactly as it does this one. The crossover sits at N=2, which fits everywhere; we do not claim growth past it.

p99.99 is not yet quotable.

A smaller jitter source remains; isolcpus / nohz_full / PREEMPT_RT is the work that closes it.

Next: kernel-enforced arbitration

A pure-userspace framework cannot stop a buggy or malicious peer from corrupting the shared ring.
A kernel mediation layer closes exactly that gap - the one thing this design cannot do from userspace.